

ЛР2 (3 пары / 6 часов)

«Модель данных и список задач в памяти. Первое “настоящее” добавление и просмотр»

Контекст проекта

Мы продолжаем **TaskTracker** из ЛР1: консольное меню уже есть. Теперь сделаем:

- модель **TaskItem** (что такое “задача” внутри программы)
 - список задач **в памяти** (пока без файла)
 - реальные команды: **Добавить** и **Показать список**
-

0) Что должно быть готово в конце ЛР2

К концу 3-й пары у студента:

✓ 1) В решении есть **2 проекта**:

- `TaskTracker.App` (консольное меню)
- `TaskTracker.Core` (модели и логика)

✓ 2) Работают функции:

- **Добавить задачу** (ввод названия, проверка что не пустое)
- **Показать список задач** (вывод всех задач с Id и статусом)

✓ 3) Есть документы:

- `docs/LR2_Report.md` (короткий отчёт: что сделано, как проверить)

✓ 4) Минимум **3 коммита** за ЛР2

1) Теория (расширенная, но только важное)

1.1. Что такое “модель данных”

Модель данных — это описание объекта “Задача” в программе: какие у неё поля.

Пример:

- **Id** — уникальный номер
- **Title** — название задачи
- **Status** — статус (Новая / В работе / Готово)

В коде это чаще всего делается как **класс** (class).

1.2. Что такое “слои проекта” и зачем два проекта

Новички часто пишут всё в одном файле — и потом сложно поддерживать.

Мы делаем **разделение**:

- **App** — **интерфейс** (меню, ввод с клавиатуры, вывод на экран)
- **Core** — **логика и модели** (что такое задача, как добавить, как хранить)

Плюс:

- легче проверять
 - легче переносить в мобильное приложение позже
 - меньше “каши” в Program.cs
-

1.3. Что такое список в памяти

Пока нет файлов и БД. Мы храним задачи только в оперативной памяти:

- Закрыв программу — задачи пропали (это нормально на этом этапе)

В C# мы используем `List<TaskItem>` — “список задач”.

1.4. Проверка ввода (очень важно)

Пользователь может ввести:

- пустую строку
- только пробелы

Наша программа **не должна падать** и **должна говорить**, что так нельзя.

Правило:

- если `Title` пустой или пробелы → задача не создаётся, выводим сообщение.

2) Практика (по кнопкам). Пара 1: создаём проект Core и модель TaskItem

Шаг 1. Открыть решение TaskTracker

1. Откройте Visual Studio
2. **File** → **Open** → **Project/Solution**
3. Выберите ваш `TaskTracker.sln` из папки репозитория

Шаг 2. Добавить новый проект `TaskTracker.Core`

1. В правой части откройте **Solution Explorer**
2. Нажмите правой кнопкой мыши на **Solution "TaskTracker"**
3. Выберите **Add** → **New Project...** (Добавить → Новый проект)
4. В поиске напишите: **Class Library**
5. Выберите:
 - **Class Library (C#)**
1. Нажмите **Next**
2. Заполните:
 - **Project name:** `TaskTracker.Core`
 - **Location:** убедитесь, что это папка вашего репозитория/solution

1. Нажмите **Next**
 2. Framework: **.NET 8.0** (или такой же как в App)
 3. Нажмите **Create**
-

Шаг 3. Удалить пустой Class1.cs

1. В проекте `TaskTracker.Core` найдите файл `Class1.cs`
 2. Правой кнопкой → **Delete** (удалить)
-

Шаг 4. Создать папку Models и класс TaskItem

4.1. Создать папку

1. Правой кнопкой по проекту `TaskTracker.Core`
2. **Add** → **New Folder**
3. Назовите папку: `Models`

4.2. Создать класс

1. Правой кнопкой по папке `Models`
 2. **Add** → **Class...**
 3. Название: `TaskItem.cs`
 4. Нажмите **Add**
-

Шаг 5. Вставить код TaskItem (модель)

Откройте `TaskItem.cs` и вставьте:

```
namespace TaskTracker.Core.Models;

public class TaskItem
{
    public int Id { get; set; }
```

```
public string Title { get; set; } = "";  
public TaskStatus Status { get; set; } = TaskStatus.New;  
}
```

Шаг 6. Создать enum статусов TaskStatus

1. Правой кнопкой по папке `Models`
2. **Add** → **Class...**
3. Название: `TaskStatus.cs`
4. **Add**
5. Удалите шаблон класса и вставьте:

```
namespace TaskTracker.Core.Models;  
  
public enum TaskStatus  
{  
    New = 0,  
    InProgress = 1,  
    Done = 2  
}
```

Шаг 7. Сохранить и собрать (Build)

1. Меню **Build** → **Build Solution**
2. Должно быть: **Build succeeded** (сборка успешна)

Шаг 8. Коммит №1 (Core добавлен)

Откройте **Git Changes**:

- сообщение коммита: `feat: add Core project with TaskItem model`
- **Commit All**

- затем **Push** (если вы пушите каждый раз)
-

3) Практика. Пара 2: подключаем Core к App и делаем список задач

Шаг 9. Подключить ссылку (Reference) из App на Core

1. В Solution Explorer откройте проект `TaskTracker.App`
 2. Найдите раздел **Dependencies**
 3. Правой кнопкой по **Dependencies** → **Add Project Reference...**
 4. Поставьте галочку **TaskTracker.Core**
 5. Нажмите **ОК**
-

Шаг 10. Откройте Program.cs и подготовьте список задач

Откройте `Program.cs` в `TaskTracker.App`.

10.1. Добавьте using

В самом верху файла добавьте:

```
using TaskTracker.Core.Models;
```

10.2. Добавьте список и счётчик Id

Сразу после using (или в начале файла) добавьте:

```
var tasks = new List<TaskItem>();  
var nextId = 1;
```

Шаг 11. Заменяем меню из ЛР1 на “меню в цикле”

Сейчас у вас меню показывается один раз. Нам надо, чтобы меню работало много раз, пока пользователь не выберет 0.

Замените содержимое Program.cs на такой вариант (полностью):

```
using TaskTracker.Core.Models;

var tasks = new List<TaskItem>();
var nextId = 1;

while (true)
{
    Console.WriteLine();
    Console.WriteLine("TaskTracker v0.2");
    Console.WriteLine("-----");
    Console.WriteLine("1) Добавить задачу");
    Console.WriteLine("2) Показать список задач");
    Console.WriteLine("0) Выход");
    Console.WriteLine("-----");
    Console.Write("Выберите пункт меню: ");

    var input = Console.ReadLine();

    if (input == "0")
    {
        Console.WriteLine("Выход...");
        break;
    }

    if (input == "1")
    {
        Console.Write("Введите название задачи: ");
        var title = Console.ReadLine() ?? "";

        // Валидация: нельзя пустое
```

```

        if (string.IsNullOrEmpty(title))
        {
            Console.WriteLine("Ошибка: название не может быть
пустым.");
            continue;
        }

        var task = new TaskItem
        {
            Id = nextId,
            Title = title.Trim(),
            Status = TaskStatus.New
        };

        nextId++;

        tasks.Add(task);
        Console.WriteLine($"Задача добавлена: #{task.Id} {tas
k.Title} [{task.Status}]");
        continue;
    }

    if (input == "2")
    {
        if (tasks.Count == 0)
        {
            Console.WriteLine("Список задач пуст.");
            continue;
        }

        Console.WriteLine("Список задач:");
        foreach (var t in tasks)
        {
            Console.WriteLine($"{t.Id}. {t.Title} [{t.Statu
s}]");
        }
    }

```

```
        continue;
    }

    Console.WriteLine("Неизвестная команда. Введите 1, 2 или 0.");
}
```

Шаг 12. Запуск и проверка

1. Нажмите **F5**
2. Проверьте сценарий:

Сценарий проверки:

- нажать 2 → "Список пуст"
- нажать 1 → ввести пусто → ошибка
- нажать 1 → ввести "Купить тетрадь" → задача добавилась
- нажать 2 → задача отображается
- нажать 0 → выход

Шаг 13. Коммит №2 (реальная логика Add/List)

Сообщение:

- `feat: add in-memory task list with add and list commands`

Commit All → Push.

4) Практика. Пара 3: маленький сервис (упрощаем Program.cs) + отчёт

Чтобы Program.cs не разрастался, вынесем логику в Core: **TaskService**.

Шаг 14. Создать папку Services и класс TaskService

1. В проекте `TaskTracker.Core` правой кнопкой
2. **Add** → **New Folder**
3. Название: `Services`
4. По папке `Services` → **Add** → **Class...**
5. Название: `TaskService.cs`

Шаг 15. Код TaskService (логика добавления и чтения)

Откройте `TaskService.cs` и вставьте:

```
using TaskTracker.Core.Models;

namespace TaskTracker.Core.Services;

public class TaskService
{
    private readonly List<TaskItem> _tasks = new();
    private int _nextId = 1;

    public TaskItem Add(string title)
    {
        if (string.IsNullOrEmpty(title))
            throw new ArgumentException("Название не может быть пустым.");

        var task = new TaskItem
        {
            Id = _nextId++,
            Title = title.Trim(),
            Status = TaskStatus.New
        };

        _tasks.Add(task);
    }
}
```

```

        return task;
    }

    public List<TaskItem> GetAll()
    {
        // Возвращаем копию, чтобы внешний код не ломал списо
к
        return _tasks.ToList();
    }
}

```

Шаг 16. Используем TaskService в Program.cs

Теперь в App мы не держим `tasks` и `nextId`, а используем сервис.

В `Program.cs`:

1. Добавьте using:

```
using TaskTracker.Core.Services;
```

1. Замените строки:

```
var tasks = new List<TaskItem>();
var nextId = 1;
```

на:

```
var service = new TaskService();
```

1. В команде "1) Добавить задачу" замените создание задачи на:

```

try
{
    var task = service.Add(title);
    Console.WriteLine($"Задача добавлена: #{task.Id} {task.Ti

```

```
tle} [{task.Status}]);  
}  
catch (ArgumentException ex)  
{  
    Console.WriteLine("Ошибка: " + ex.Message);  
}
```

1. В команде "2) Показать список задач" замените `tasks` на:

```
var tasks = service.GetAll();
```

| Главное: программа должна работать как раньше, только код стал чище.

Шаг 17. Проверка после изменений

Запустите **F5** и повторите сценарий:

- add пустой → ошибка
- add нормальный → добавилось
- list → показывает

Шаг 18. Отчёт `docs/LR2_Report.md`

Создайте файл `docs/LR2_Report.md` и вставьте:

```
# Отчёт ЛР2 – TaskTracker  
  
## Что сделано  
- Добавлен проект TaskTracker.Core  
- Создана модель TaskItem и enum TaskStatus  
- Реализован TaskService (Add/GetAll)  
- В TaskTracker.App добавлены команды:  
  - Добавить задачу  
  - Показать список
```

Как проверить

- 1) Запустить программу (F5)
- 2) Нажать 2 → список пуст
- 3) Нажать 1 и ввести пусто → ошибка
- 4) Нажать 1 и ввести "Купить тетрадь" → задача добавлена
- 5) Нажать 2 → задача отображается

Шаг 19. Коммит №3 (сервис + отчёт)

Сообщение:

- `feat: add TaskService and LR2 report`

Commit All → Push.

5) Что сдавать (чек-лист)

Студент отправляет:

1. ☒ ссылку на репозиторий
2. ☒ скрин/видео, где:
 - добавили задачу
 - показали список
3. ☒ в репозитории есть:
 - `TaskTracker.Core` (Models + Services)
 - `docs/LR2_Report.md`
 - минимум 3 коммита за ЛР2

6) Частые ошибки и как исправить

Ошибка: "не видит TaskTracker.Core"

Проверьте:

- `TaskTracker.App` → `Dependencies` → `Add Project Reference` → `TaskTracker.Core`

Ошибка: "List<> не найден"

Сверху добавьте:

```
using System.Collections.Generic;
```

(или включите "Implicit usings" — чаще всего в новых проектах уже включено)

Ошибка: "программа сразу закрывается"

Запускайте **F5**, а не просто открывайте exe.

Или добавьте в конце:

```
Console.ReadLine();
```

(но с циклом `while(true)` обычно не нужно)

7) Если не на C# (краткая адаптация требований)

Главное: модель задачи + список в памяти + команды add/list + отчёт.

- Python: класс Task + список `tasks = []`
- Java: класс Task + `ArrayList<Task>`
- Kotlin: data class + MutableList

Сдача одинаковая: репо + README + docs/LR2_Report.md + демо add/list.
